

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-SCENARIO BASED ASSESSMENT
Question	Q2 — Scenario 2: Comprehensive Password Strength Checker (20 Marks)

Scenario 2: Comprehensive Password Strength Checker [20 Marks]

A cybersecurity system requires strict password validation for user accounts. Passwords are provided as strings with varying lengths and character combinations. The system must evaluate the presence of uppercase, lowercase, digits, and special symbols. It should also analyse patterns to detect weak or predictable passwords. Based on multiple criteria, passwords should be categorized into strength levels (Weak, Moderate, Strong, Very Strong). Control flow constructs must be used to apply layered validation rules. The evaluation logic should be encapsulated within reusable functions. The program should provide meaningful feedback for improvement.

Task

Design a Python solution to implement this comprehensive password strength checker with feedback, demonstrating the following key skills:

- Character-level string analysis using loops
- Checking uppercase, lowercase, digit, symbol presence
- Multi-level classification using if-elif-else
- Functions for validation and feedback generation
- Handling edge cases: empty or very long passwords

Approach / Design

- **Character analysis (analyze_characters):** loops over every character of the password and tallies uppercase, lowercase, digit, and special-symbol counts.
- **Pattern detection (has_weak_pattern):** checks for common weak words (e.g. 'password', 'qwerty'), repeated-character runs, and ascending numeric sequences.
- **Criteria evaluation (evaluate_criteria):** combines length checks and character-type checks into a single dictionary of boolean rules.
- **Layered classification (classify_strength):** applies if-elif-else rules on length, character variety, and pattern safety to assign Weak, Moderate, Strong, or Very Strong.
- **Feedback generation (generate_feedback):** builds a targeted list of improvement tips based on exactly which criteria failed.
- **Edge-case handling (check_password):** explicitly handles empty passwords and passwords exceeding a maximum length before running the full pipeline.
- **Batch handling (check_all_passwords):** loops over any number of passwords, so the design scales to bulk validation.

Python Program

```
"""
C02-AT2 - Scenario Based Assessment
Scenario 2: Comprehensive Password Strength Checker
-----
A cybersecurity system requires strict password validation.
This program:
1. Analyzes each character of a password using loops
2. Checks for uppercase, lowercase, digit, and special-symbol presence
3. Detects weak/predictable patterns
4. Classifies strength into Weak / Moderate / Strong / Very Strong
   using if-elif-else layered rules
5. Generates meaningful improvement feedback
6. Handles edge cases (empty / very long passwords)
7. Is fully modularized using functions
"""

import string

MIN_LENGTH = 8
STRONG_LENGTH = 12

SPECIAL_CHARS = string.punctuation

# Common weak/predictable passwords and sequences to flag
COMMON_WEAK_PATTERNS = [
    "password", "12345", "123456", "qwerty", "letmein",
    "admin", "welcome", "abc123", "iloveyou"
]

# -----
# 1. Character-level analysis function
# -----
def analyze_characters(password):
    """
    Loops through each character of the password and counts how
    many uppercase, lowercase, digit, and special-symbol
    characters it contains.
    """
```

```

counts = {"upper": 0, "lower": 0, "digit": 0, "special": 0}

for char in password:
    if char.isupper():
        counts["upper"] += 1
    elif char.islower():
        counts["lower"] += 1
    elif char.isdigit():
        counts["digit"] += 1
    elif char in SPECIAL_CHARS:
        counts["special"] += 1

return counts

# -----
# 2. Pattern / predictability detection function
# -----
def has_weak_pattern(password):
    """
    Detects common weak/predictable substrings (case-insensitive)
    and simple repeated-character or sequential-digit patterns.
    """
    lowered = password.lower()

    for pattern in COMMON_WEAK_PATTERNS:
        if pattern in lowered:
            return True

    # Repeated character run, e.g. "aaaa" or "1111"
    for i in range(len(password) - 3):
        if password[i] == password[i + 1] == password[i + 2] == password[i + 3]:
            return True

    # Simple ascending numeric sequence, e.g. "1234"
    digits = "0123456789"
    for i in range(len(digits) - 3):
        if digits[i:i + 4] in password:
            return True

    return False

# -----
# 3. Criteria evaluation function
# -----
def evaluate_criteria(password):
    """
    Evaluates a password against each individual validation rule
    and returns a dictionary of booleans.
    """
    counts = analyze_characters(password)

    return {
        "length_ok": len(password) >= MIN_LENGTH,
        "long_enough_for_strong": len(password) >= STRONG_LENGTH,
        "has_upper": counts["upper"] > 0,
        "has_lower": counts["lower"] > 0,
        "has_digit": counts["digit"] > 0,
        "has_special": counts["special"] > 0,
        "no_weak_pattern": not has_weak_pattern(password),
    }

# -----
# 4. Strength classification function (layered if-elif-else)
# -----
def classify_strength(criteria):
    """

```

Applies layered validation rules using if-elif-else to assign a strength level: Weak, Moderate, Strong, or Very Strong.

```
"""
variety_score = sum([
    criteria["has_upper"],
    criteria["has_lower"],
    criteria["has_digit"],
    criteria["has_special"],
])

if not criteria["length_ok"] or not criteria["no_weak_pattern"]:
    return "Weak"
elif criteria["length_ok"] and variety_score <= 2:
    return "Moderate"
elif criteria["length_ok"] and variety_score == 3:
    return "Strong"
elif criteria["long_enough_for_strong"] and variety_score == 4:
    return "Very Strong"
else:
    return "Strong"
```

```
# -----
# 5. Feedback generation function
# -----
def generate_feedback(criteria):
    """
    Builds a list of specific, actionable feedback messages based
    on which criteria failed.
    """
    feedback = []

    if not criteria["length_ok"]:
        feedback.append(f"Use at least {MIN_LENGTH} characters.")
    elif not criteria["long_enough_for_strong"]:
        feedback.append(f"Use {STRONG_LENGTH}+ characters for maximum strength.")

    if not criteria["has_upper"]:
        feedback.append("Add at least one uppercase letter.")
    if not criteria["has_lower"]:
        feedback.append("Add at least one lowercase letter.")
    if not criteria["has_digit"]:
        feedback.append("Add at least one digit.")
    if not criteria["has_special"]:
        feedback.append("Add at least one special character (e.g. ! @ # $).")
    if not criteria["no_weak_pattern"]:
        feedback.append("Avoid common words and predictable/repeated patterns.")

    if not feedback:
        feedback.append("Great job! This password meets all strength criteria.")

    return feedback
```

```
# -----
# 6. Single password check function (handles edge cases)
# -----
def check_password(password):
    """
    Runs the full validation pipeline on one password and returns
    a result dictionary. Handles edge cases such as an empty
    string or an extremely long password.
    """
    if password is None or len(password) == 0:
        return {
            "password": password,
            "strength": "Invalid",
            "feedback": ["Password cannot be empty."],
        }
```

```

    if len(password) > 128:
        return {
            "password": password[:20] + "...",
            "strength": "Invalid",
            "feedback": ["Password exceeds the maximum allowed length (128 characters)."],
        }

    criteria = evaluate_criteria(password)
    strength = classify_strength(criteria)
    feedback = generate_feedback(criteria)

    return {
        "password": password,
        "strength": strength,
        "feedback": feedback,
    }

# -----
# 7. Batch processing function (handles multiple passwords)
# -----
def check_all_passwords(password_list):
    """
    Iterates through a list of passwords and checks each one,
    returning a list of result dictionaries. Scales to any
    number of passwords.
    """
    results = []
    for pwd in password_list:
        results.append(check_password(pwd))
    return results

# -----
# 8. Report/display function
# -----
def print_report(results):
    print("=" * 70)
    print("PASSWORD STRENGTH CHECKER REPORT")
    print("=" * 70)

    for idx, res in enumerate(results, start=1):
        display_pwd = res["password"] if res["password"] else "(empty)"
        print(f"\nPassword #{idx}: {display_pwd}")
        print(f"  Strength: {res['strength']}")
        print("  Feedback:")
        for msg in res["feedback"]:
            print(f"    - {msg}")

# -----
# 9. Main driver
# -----
def main():
    test_passwords = [
        "", # edge case: empty
        "abc123", # too short, weak
        "password123", # common weak pattern
        "Sunshine1", # moderate
        "Sunshine@123", # strong
        "Tr0ub4dor&3xample!", # very strong
        "aaaa1111", # repeated pattern -> weak
    ]

    results = check_all_passwords(test_passwords)
    print_report(results)

if __name__ == "__main__":

```

```
main()
```

Sample Output

```
=====
PASSWORD STRENGTH CHECKER REPORT
=====

Password #1: (empty)
Strength: Invalid
Feedback:
  - Password cannot be empty.

Password #2: abc123
Strength: Weak
Feedback:
  - Use at least 8 characters.
  - Add at least one uppercase letter.
  - Add at least one special character (e.g. ! @ # $).
  - Avoid common words and predictable/repeated patterns.

Password #3: password123
Strength: Weak
Feedback:
  - Use 12+ characters for maximum strength.
  - Add at least one uppercase letter.
  - Add at least one special character (e.g. ! @ # $).
  - Avoid common words and predictable/repeated patterns.

Password #4: Sunshine1
Strength: Strong
Feedback:
  - Use 12+ characters for maximum strength.
  - Add at least one special character (e.g. ! @ # $).

Password #5: Sunshine@123
Strength: Very Strong
Feedback:
  - Great job! This password meets all strength criteria.

Password #6: Tr0ub4dor&3xample!
Strength: Very Strong
Feedback:
  - Great job! This password meets all strength criteria.

Password #7: aaaa1111
Strength: Weak
Feedback:
  - Use 12+ characters for maximum strength.
  - Add at least one uppercase letter.
  - Add at least one special character (e.g. ! @ # $).
  - Avoid common words and predictable/repeated patterns.
```

Conclusion

The program validates passwords through layered, function-based logic: character-level loops gather composition statistics, pattern checks catch predictable passwords, and if-elif-else rules combine these signals into a clear strength rating. Empty and overly long inputs are handled as explicit edge cases, and actionable feedback is generated for every password so users know exactly how to improve it. Because each password is processed independently inside a loop over a list, the checker scales to validating any number of passwords.